

ES98A Assignment 3

Author: Jabir Hussain (2007942)

Exercise 3.1.

We consider the log-uniform model κ where

$$a(x) = \sum_{k \in \mathbb{N} \cup 0} \xi_k \cos(kx) + \sum_{k \in \mathbb{N}} \eta_k \sin(kx)$$

with uniformly distributed real, random, independent coefficients $\xi_k, \eta_k \sim U([-(1+k)^{-4}, (1+k)^{-4}])$ where $U([a, b])$ denotes the uniform probability distribution on interval $[a, b]$. We want to reformulate $a(x)$ as a Fourier series in the form

$$a(x) = \sum_{k \in \mathbb{Z}} a_k \frac{e^{ikx}}{\sqrt{2\pi}}$$

We can rewrite the summations by using the exponential forms of the trigonometric functions:

$$\cos(z) = \frac{e^{iz} + e^{-iz}}{2}$$

$$\sin(z) = \frac{e^{iz} - e^{-iz}}{2i}$$

We use these formulae to express the trigonometric functions in $a(x)$:

$$\xi_k \cos(kx) = \frac{\xi_k}{2} (e^{ikx} + e^{-ikx})$$

$$\eta_k \sin(kx) = \frac{\eta_k}{2i} (e^{ikx} - e^{-ikx})$$

Collecting like-terms gives us

$$\text{coefficient of } e^{ikx} : \frac{\xi_k}{2} + \frac{\eta_k}{2i}$$

$$\text{coefficient of } e^{-ikx} : \frac{\xi_k}{2} - \frac{\eta_k}{2i}$$

This gives the expression

$$a(x) = \xi_0 \sum_{k \geq 1} \left(\frac{\xi_k}{2} + \frac{\eta_k}{2i} \right) e^{ikx} + \sum_{k \geq 1} \left(\frac{\xi_k}{2} - \frac{\eta_k}{2i} \right) e^{-ikx}.$$

Matching coefficients of e^{ikx} gives, for $k \geq 1$,

$$\frac{a_k}{\sqrt{2\pi}} = \frac{\xi_k}{2} + \frac{\eta_k}{2i}, \quad \frac{a_{-k}}{\sqrt{2\pi}} = \frac{\xi_k}{2} - \frac{\eta_k}{2i},$$

and for $k = 0$,

$$\frac{a_0}{\sqrt{2\pi}} = \xi_0.$$

Hence

$$a_0 = \sqrt{2\pi} \xi_0, \quad a_k = \frac{\sqrt{2\pi}}{2} \left(\xi_k + \frac{\eta_k}{i} \right), \quad a_{-k} = \frac{\sqrt{2\pi}}{2} \left(\xi_k - \frac{\eta_k}{i} \right).$$

Recall the definition of the H^1 -norm in terms of the Fourier coefficients:

$$\|a\|_{H^1}^2 = \sum_{k \in \mathbb{Z}} (1 + k^2) |a_k|^2.$$

To show that $a \in H^1_{\text{per}}(\mathbb{T}; \mathbb{R})$, this series must converge.

For $k \geq 1$ we have

$$a_k = \frac{\sqrt{2\pi}}{2} \left(\xi_k + \frac{\eta_k}{i} \right).$$

By the triangle inequality,

$$|a_k| \leq \frac{\sqrt{2\pi}}{2} (|\xi_k| + |\eta_k|).$$

In the log-uniform model, it is shown that $|\xi_k|, |\eta_k| \leq (1+k)^{-4}$.

$$|a_k| \leq C(1+k)^{-4}$$

where $C = \sqrt{2\pi}$.

Thus,

$$(1+k^2)|a_k|^2 \leq C^2(1+k^2)(1+k)^{-8} \implies C^2 k^{-6} \quad \text{as } k \rightarrow \infty.$$

Since $\sum_{k=1}^{\infty} k^{-p}$ converges for $p > 1$, the series $\sum_{k \in \mathbb{Z}} (1+k^2)|a_k|^2$ converges. Thus $\|a\|_{H^1} < \infty$ for every choice of the coefficients, and so

$$a \in H_{\text{per}}^1(\mathbb{T}; \mathbb{R})$$

Exercise 3.2.

In this exercise, we maintain that κ is a log-normal model where

$$a(x) = \sum_{k \in \mathbb{N} \cup 0} \xi_k \cos(kx) + \sum_{k \in \mathbb{N}} \eta_k \sin(kx)$$

The coefficients are now Gaussian, where $\xi_k, \eta_k \sim \mathcal{N}(0, (1+k)^{-4})$

We can represent $a(x)$ in the form

$$a(x) = \sum_{k \in \mathbb{Z}} a_k \frac{e^{ikx}}{\sqrt{2\pi}}.$$

We also have the H^1 -norm

$$\|a\|_{H^1}^2 = \sum_{k \in \mathbb{Z}} (1+k^2) |a_k|^2$$

In order to show that $a \in H^1$, the random series

$$\sum_{k \in \mathbb{Z}} (1+k^2) |a_k|^2$$

must converge.

From Exercise 3.1., we know that $\exists C > 0$ where $\forall k > 1$

$$|a_k| \leq C(|\xi_k| + |\eta_k|)$$

by the triangle inequality. Since we have independent random variables, we can apply Kolmogorov's Two Series theorem, which is as follows:

Kolmogorov's Two Series theorem:

Suppose that $x_n, n \in \mathbb{N}$, are independent real-valued random variables, and that

$$(1) : \sum_{n \in \mathbb{N}} |\mathbb{E}[x_n]| < \infty$$

and

$$(2) : \sum_{n \in \mathbb{N}} \mathbb{E}[x_n^2] < \infty.$$

Then,

$$\mathbb{P}[\sum_{n \in \mathbb{N}} x_n \text{ converges}] = 1$$

Given that for a real-valued function $|a_k| = |a_{-k}|$, we can write the H^1 -norm as

$$\|a\|_{H^1}^2 = |a_0|^2 + 2 \sum_{k=1}^{\infty} (1 + k^2) |a_k|^2.$$

For $k \geq 1$, we define Y_k such that

$$Y_k := (1 + k^2) |a_k|^2.$$

From this, the following holds true:

- $Y_k \geq 0$
- Each Y_k is independent as each ξ_k, η_k is independent for each k

$$\sum_{k=1}^{\infty} Y_k = \sum_{k=1}^{\infty} (1 + k^2) |a_k|^2$$

To show that $\sum_k Y_k$ converges, we must apply Kolmogorov's theorem and show that the series (1) and (2) from above hold.

(1):

We use the bound

$$|a_k| \leq C(|\xi_k| + |\eta_k|).$$

Taking the square and applying Minkowski's inequality for $p = 2$: $|x + y|^2 \leq 2(x^2 + y^2)$:

$$|a_k|^2 \leq 2C^2(|\xi_k|^2 + |\eta_k|^2).$$

By taking the expectations of the inequality, we get

$$\mathbb{E}|a_k|^2 \leq 2C^2(\mathbb{E}|\xi_k|^2 + \mathbb{E}|\eta_k|^2)$$

Given that $\xi_k, \eta_k \sim \mathcal{N}(0, (1 + k)^{-4})$, it follows that

$$\mathbb{E}|\xi_k|^2 = \mathbb{E}|\eta_k|^2 = (1 + k)^{-4}$$

Substituting this in gives:

$$\begin{aligned} \mathbb{E}|a_k|^2 &\leq 2C^2((1 + k)^{-4} + (1 + k)^{-4}) \\ &\implies \leq 4C^2(1 + k)^{-4} \end{aligned}$$

We use the definition $Y_k := (1 + k^2) |a_k|^2$ and multiply through accordingly:

$$\mathbb{E}[Y_k] = (1 + k^2) \mathbb{E}|a_k|^2 \leq 4C^2(1 + k^2)(1 + k)^{-4}$$

As $k \rightarrow \infty$,

$$\mathbb{E}[Y_k] \lesssim k^2 k^{-4} = k^{-2}.$$

Given that $\sum_{k=1}^{\infty} k^{-p}$ converges for $p > 1$ and $2 > 1$, we can conclude that the first series is convergent:

$$\sum_{k=1}^{\infty} \mathbb{E}[Y_k] < \infty$$

(2)

Now we must show the second series converges. By squaring Y_k we get the form

$$Y_k^2 = (1 + k^2)^2 |a_k|^4.$$

Recall a_k satisfies

$$|a_k| \leq C(|\xi_k| + |\eta_k|).$$

We raise the expression to the power of 4 and apply Minkowski's inequality, yielding $x = C\xi_k, y = C\eta_k$:

$$|a_k|^4 \leq 8C^4(|\xi_k|^4 + |\eta_k|^4).$$

We then take expectations on this equality:

$$\mathbb{E}|a_k|^4 \leq 8C^4(\mathbb{E}|\xi_k|^4 + \mathbb{E}|\eta_k|^4).$$

By the fourth moment/kurtosis of a normal random variable, we have $u \sim N(0, \sigma^2)$, $\mathbb{E}|u|^4 = 3\sigma^4$. Given that $\sigma^2 = (1+k)^{-4}$, $\mathbb{E}|\xi_k|^4 = 3(\sigma_k^2)^2$

$$\implies \mathbb{E}|\xi_k|^4 = 3(1+k)^{-8}, \quad \mathbb{E}|\eta_k|^4 = 3(1+k)^{-8}.$$

Substituting the fourth-moment formulas into the previous inequality gives

$$\mathbb{E}|a_k|^4 \leq 8C^4(3(1+k)^{-8} + 3(1+k)^{-8}) = 48C^4(1+k)^{-8}.$$

By the definition of $Y_k = (1+k^2)|a_k|^2$,

$$\mathbb{E}[Y_k^2] = (1+k^2)^2 \mathbb{E}|a_k|^4 \leq 48C^4(1+k^2)^2(1+k)^{-8}.$$

As $k \rightarrow \infty$,

$$\mathbb{E}[Y_k^2] \lesssim k^4 \cdot k^{-8} = k^{-4},$$

and again $\sum_{k=1}^{\infty} k^{-4}$ converges (since $4 > 1$). Thus

$$\sum_{k=1}^{\infty} \mathbb{E}[Y_k^2] < \infty \quad \Rightarrow \quad \sum_{k=1}^{\infty} \text{Var}(Y_k) < \infty.$$

We can conclude that the second series is convergent:

$$\sum_{k=1}^{\infty} \mathbb{E}[Y_k^2] < \infty$$

Since we have shown

$$(1) : \sum_{n \in \mathbb{N}} |\mathbb{E}[x_n]| < \infty$$

and

$$(2) : \sum_{n \in \mathbb{N}} \mathbb{E}[x_n^2] < \infty.$$

In turn,

$$\mathbb{P}\left[\sum_{n \in \mathbb{N}} x_n \text{ converges}\right] = 1$$

By the definition of the Sobolev space, this is exactly

$$\mathbb{P}[a \in H_{\text{per}}^1(\mathbb{T}; \mathbb{R})] = 1.$$

Exercise 3.3.

In this exercise, we will draw vanilla Monte Carlo approximate samples of a , truncating the sums to $|k| \leq k_{\max} = 40$. Firstly, we import the necessary modules for linear algebra and data visualisation.

```
In [118... # import necessary libraries
import numpy as np # linear algebra
import matplotlib.pyplot as plt # plot
%config InlineBackend.figure_format = 'retina'

np.random.seed(2007942) # set seed for reproducibility

k_max = 40
modes = np.arange(k_max + 1, dtype=float) # Fourier modes
variance = (1 + modes)**(-4)
stdev = np.sqrt(variance)

N = 2000 # number of grid points
grid = np.linspace(0, 2*np.pi, N, endpoint=False)
```

```
mc_samples = 5
```

We then write two functions, one for the log-uniform model, and one for the log-normal model. We recall that

$$a(x) = \sum_{k=0}^{k_{max}} \xi_k \cos(kx) + \sum_{k=0}^{k_{max}} \eta_k \sin(kx)$$

```
In [119... def log_uniform(x, modes):
    # sanity checks to ensure dimensions and values are as intended
    assert isinstance(x, np.ndarray), "x must be a NumPy array"
    assert x.ndim == 1, "x must be a 1D array"
    assert isinstance(modes, np.ndarray), "modes must be a NumPy array"
    assert modes.ndim == 1, "modes must be 1D"
    assert modes[0] == 0, "modes must start at 0"
    assert np.all(modes >= 0), "modes must be non-negative integers"
    assert len(modes) >= 2, "Need at least modes 0 and 1"

    widths = (1 + modes)*(-4)

    xi = np.random.uniform(-widths, widths)
    eta = np.random.uniform(-widths, widths)

    cos_matrix = np.cos(np.outer(modes, x))
    sin_matrix = np.sin(np.outer(modes[1:], x))

    a_x = (xi[:, None] * cos_matrix).sum(axis=0) + (eta[1:, None] * sin_matrix).sum(axis=0)

    return a_x

def log_normal(x, modes, std_dev):
    # sanity checks to ensure dimensions and values are as intended
    assert isinstance(x, np.ndarray), "x must be a NumPy array"
    assert x.ndim == 1, "x must be 1D"
    assert isinstance(modes, np.ndarray), "modes must be a NumPy array"
    assert modes.ndim == 1, "modes must be 1D"
    assert modes[0] == 0, "modes must start at 0"
    assert np.all(modes >= 0), "modes must be non-negative"
    assert isinstance(std_dev, np.ndarray), "std_dev must be a NumPy array"
    assert std_dev.shape == modes.shape, "std_dev must match modes shape"

    xi = np.random.normal(0.0, std_dev)
    eta = np.random.normal(0.0, std_dev)

    cos_matrix = np.cos(np.outer(modes, x))
    sin_matrix = np.sin(np.outer(modes[1:], x))

    a_x = (xi[:, None] * cos_matrix).sum(axis=0) + (eta[1:, None] * sin_matrix).sum(axis=0)

    return a_x
```

We iterate over the number of Monte Carlo samples (5) and draw samples of a and $\kappa(a)$ from the log-uniform and log-normal distributions.

```
In [120... log_uniform_samples = []
log_normal_samples = []
kappa_uniform_samples = []
kappa_normal_samples = []

for _ in range(mc_samples):
    a_uniform = log_uniform(grid, modes)
    a_normal = log_normal(grid, modes, stdev)

    log_uniform_samples.append(a_uniform)
    log_normal_samples.append(a_normal)

    kappa_uniform_samples.append(np.exp(a_uniform))
    kappa_normal_samples.append(np.exp(a_normal))
```

The realisations are plotted using a helper function to reduce repeated code.

```
In [121... def plot_realisations(x, samples, title, ylabel):
    plt.figure(figsize=(8,4))
    for sample in samples:
        plt.plot(x, sample, lw=1)
    plt.title("2007942 -" + title)
    plt.xlabel("x")
```

```

plt.ylabel(ylabel)
plt.grid(True)
plt.tight_layout()

plot_realisations(grid, log_uniform_samples,
                  " Five realisations of log-uniformly distributed  $a(x)$ ",
                  "a(x)")

fig1a = r"Figure 1a. Five log-uniform realisations of  $a(x)$  where they show smooth and small amplitudes"
plt.figtext(0.5, -0.1, fig1a, wrap=True, ha='center', fontsize=12)

plot_realisations(grid, log_normal_samples,
                  " Five realisations of log-normally distributed  $a(x)$ ",
                  "a(x)")

fig1b = r"Figure 1b. Five log-normal realisations of  $a(x)$  where they show rougher curves with larger amplitudes"
plt.figtext(0.5, -0.1, fig1b, wrap=True, ha='center', fontsize=12)

plot_realisations(grid, kappa_uniform_samples,
                  " Five realisations of log-uniform  $\kappa(x)$ ",
                  " $\kappa(x)$ ")

fig1c = r"Figure 1c. Five log-uniform realisations of  $e^{a(x)}$  where they show slight exponential stretching"
plt.figtext(0.5, -0.1, fig1c, wrap=True, ha='center', fontsize=12)

plot_realisations(grid, kappa_normal_samples,
                  " Five realisations of log-normal  $\kappa(x)$ ",
                  " $\kappa(x)$ ")

fig1d = r"Figure 1d. Five log-normal realisations of  $e^{a(x)}$  where they show significant exponential stretching"
plt.figtext(0.5, -0.1, fig1d, wrap=True, ha='center', fontsize=12)

plt.show()

```

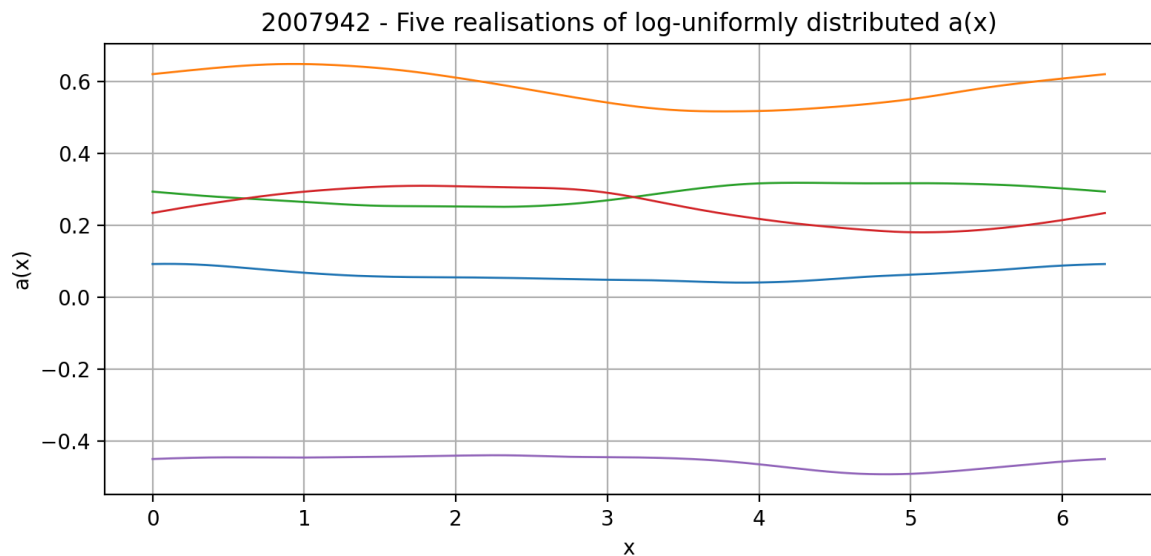


Figure 1a. Five log-uniform realisations of $a(x)$ where they show smooth and small amplitudes.

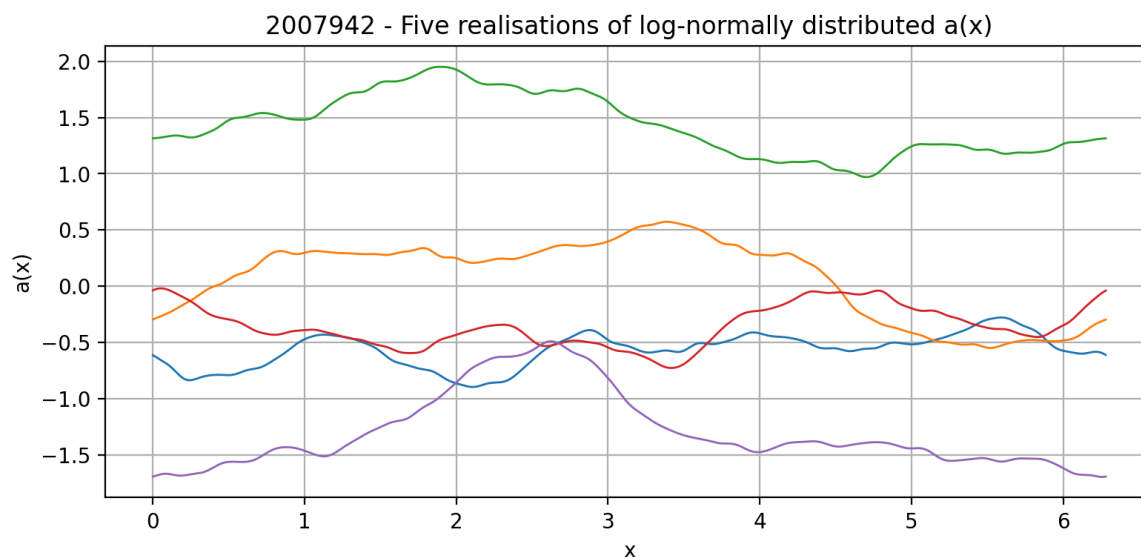


Figure 1b. Five log-normal realisations of $a(x)$ where they show rougher curves with larger amplitudes.

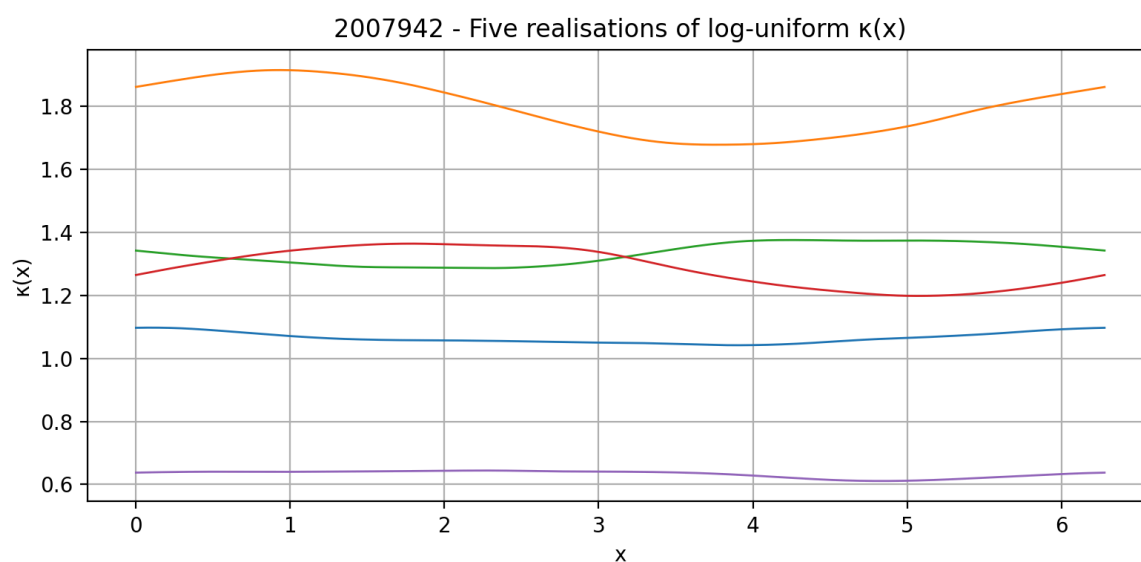


Figure 1c. Five log-uniform realisations of $e^{a(x)}$ where they show slight exponential stretching.

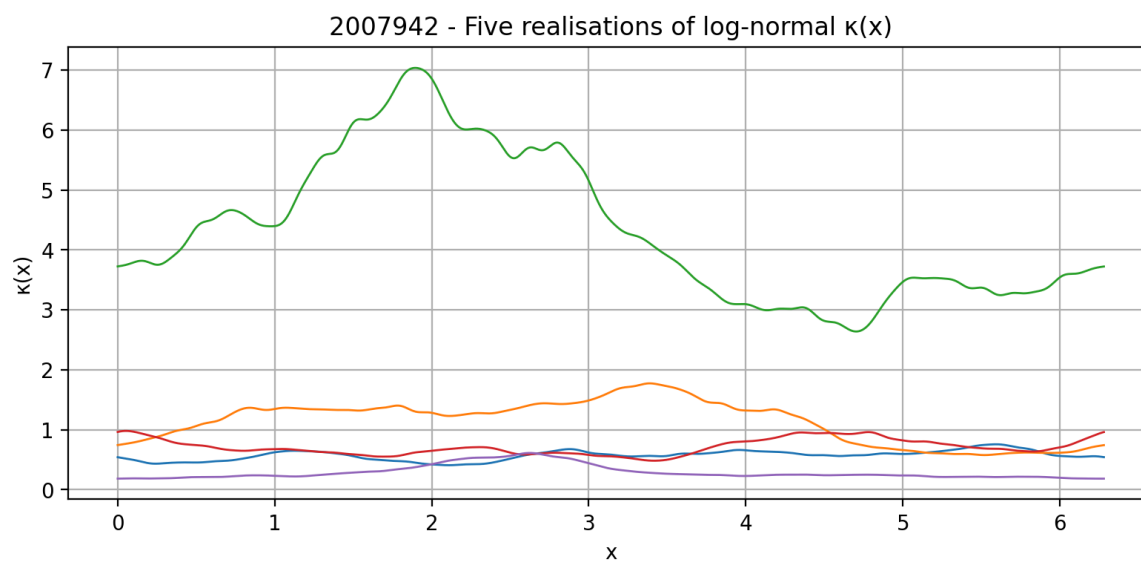


Figure 1d. Five log-normal realisations of $e^{a(x)}$ where they show significant exponential stretching.

From these plots, we can evaluate the following:

Plots $a(x)$ **against** x :

For similarities, both the log-normal and log-uniform plots take both positive and negative values, with no discontinuities. However, for differences, there are several:

Firstly, across the realisation plots for log-uniform and log-normal $a(x)$, we see a significant difference in both amplitude and smoothness. The log-uniform plot values are bounded since each Fourier coefficient is drawn from a fixed finite interval of $[-(1+k)^{-4}, (1+k)^{-4}]$. In turn, the variance is bounded and all the sample curves have little dispersion between them. The curves are also visually smooth, with seemingly similar amplitudes.

Conversely, the log-normal plot has much greater variations in amplitude, with some curves (i.e. the blue curve) showing modest magnitude. Some of the curves have much more pronounced peaks. These curves are visually rougher than the log-uniform plot as a result of the Gaussian Fourier coefficients having unbounded tails. This means that coefficients are occasionally large, amplifying the curves which in turn produce roughness. There is also a greater dispersion between the curves in the log-normal plot. The Gaussian Fourier coefficients have unbounded variance, meaning some realisations of $a(x)$ can deviate significantly beyond the mean. In turn, some curves will be considerably further than the others, such as the green curve in Figure 1b.

Plots $\kappa(x)$ **against** x

Similar to the plots of $a(x)$, both the log-normal and log-uniform $\kappa(x) = e^{a(x)}$ plots are strictly positive with no discontinuities. Like in the case of the $a(x)$ plots, there are significant differences between the plots for $\kappa(x)$.

The log-uniform curves are more dispersed than the log-normal plot, as the uniform $a(x)$ values have apart from the purple curve are positive. When exponentiating, the values are expanded even in small perturbations, leading to greater values of $\kappa(x)$. Conversely, with the majority of log-normal $a(x)$ curves falling under 0, the exponentiation compresses these numbers into a narrow band in the log-normal $\kappa(x)$ plot. The green curve from the log-normal $a(x)$ curve is strictly positive, and this is further highlighted in its curve on the $\kappa(x)$ curve, as these positive values are expanded under exponentiation.

Exercise 3.4.

In this exercise, we will numerically solve the ODE

$$\begin{aligned}\dot{x}(t) &= \kappa(x(t)) & \text{for } t > 0 \\ &= e^{a(x(t))} \\ x(0) &= 1\end{aligned}$$

This numerical solver will use an Euler scheme with constant time step size 0.01, solving the system for $t \geq 0$ up until the first time t where $x(t) \geq x_* := 6$ and at least up until $t = \frac{1}{5}$. For each realisation of $a(x)$ that is generated, the program must also generate $\kappa(x)$ and trajectory $x(t)$.

We firstly create a helper function to build the field from discrete samples by interpolation. `scipy`'s `'interp1d'` function allows us to use the discrete samples and generate a continuous approximation of $a(x)$, necessary for the ODE solver.

```
In [122...] from scipy.interpolate import interp1d

def build_field(grid, values):
    # sanity checks
    assert grid.ndim == 1, "grid must be 1-dimensional"
    assert values.shape == grid.shape, "values must have the same shape as grid"
    return interp1d(grid, values, kind='linear', fill_value='extrapolate', assume_sorted=True)
```

We then create the function `ode_solver` that implements the explicit Euler method described above. It takes the form

$$x_{n+1} = x_n + \Delta t \kappa(x_n)$$

. For each iteration, the code checks if any of the boundary criterion (minimum $t = 0.2$, stops at $t = 6$).

```
In [123...] def ode_solver(kappa_field, x_0 = 1.0, dt = 0.01, t_min = 0.2, x_star = 6.0):
    # sanity checks
    assert callable(kappa_field), "kappa_field must be a callable function"
    assert dt > 0, "dt must be positive"
    assert t_min >= 0, "t_min must be non-negative"

    ts = [0.0]
```



```

xs = [x_0]

while True:
    # get most recent time and x value
    t_n = ts[-1]
    x_n = xs[-1]

    # compute next step using explicit Euler method
    kappa = np.exp(kappa_field(x_n))
    x_next = x_n + dt * kappa
    t_next = t_n + dt

    ts.append(t_next)
    xs.append(x_next)

    # check boundary conds.
    if x_next >= x_star and t_next >= t_min:
        break
    if t_next >= t_min and x_next < x_star:
        continue
    if x_next >= x_star:
        break

return np.array(ts), np.array(xs)

```

The `sample_generator` helper function is used to generate the uniform and normal fields used for `ode_solver` for 5 realisations. Whilst it could have been called directly in the list comprehension (i.e. `[build_field(grid, log_uniform(grid, modes))]`), this obfuscated approach would prove difficult for debugging and readability.

```

In [124... def sample_generator(log):
    if log == 'uniform':
        samples = log_uniform(grid, modes)
        return build_field(grid, samples)
    elif log == 'normal':
        samples = log_normal(grid, modes, stdev)
        return build_field(grid, samples)

uniform_fields = [sample_generator('uniform') for _ in range(mc_samples)]
normal_fields = [sample_generator('normal') for _ in range(mc_samples)]

```

The `simulate` function solves the ODE for each realisation sample.

```

In [125... def simulate(fields):
    trajectories, breakout_times = [], []
    for field in fields:
        t, x = ode_solver(field)
        trajectories.append((t, x))
        breakout_times.append(t[-1])
    return trajectories, breakout_times

uniform_trajectories, uniform_breakout_times = simulate(uniform_fields)
normal_trajectories, normal_breakout_times = simulate(normal_fields)

```

We plot each trajectory as follows

```

In [126... colors = ["blue", "orange", "green", "red", "purple"]

def plot_trajectories(trajs, title):
    plt.figure(figsize=(8,4))
    for (t, x), c in zip(trajs, colors):
        plt.plot(t, x, color=c)
    plt.xlabel("t (s)")
    plt.ylabel("x(t)")
    plt.title("2007942 - " + title)
    plt.grid(True)
    plt.show()

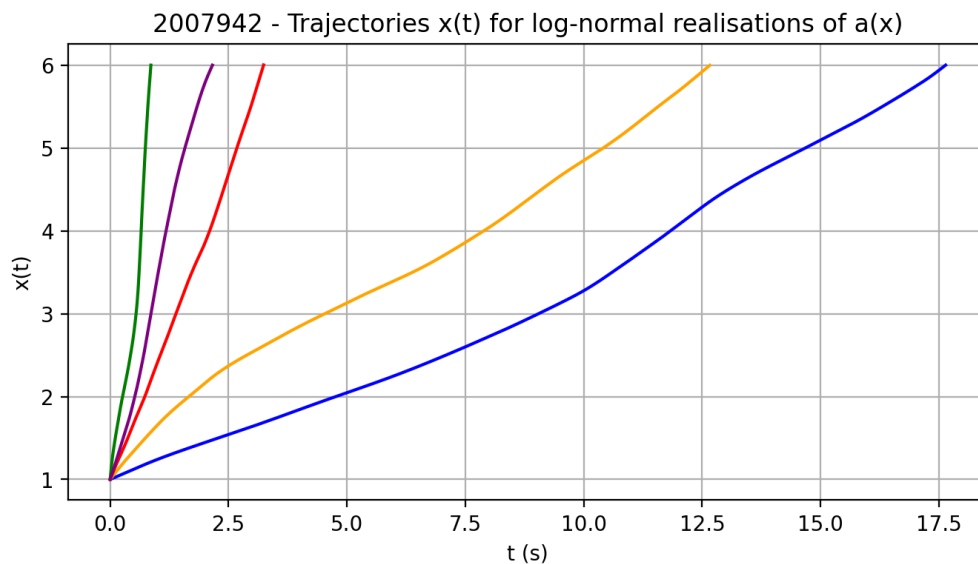
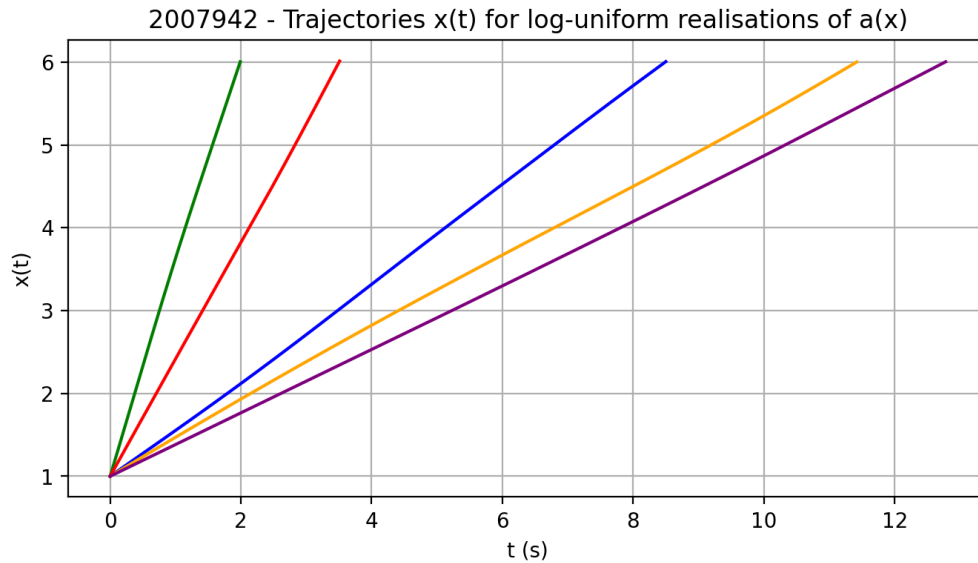
plot_trajectories(uniform_trajectories,
                  "Trajectories x(t) for log-uniform realisations of a(x)")

plot_trajectories(normal_trajectories,
                  "Trajectories x(t) for log-normal realisations of a(x)")

# print breakout times
print("Breakout times t* for log-uniform model:")
for i, tstar in enumerate(uniform_breakout_times):
    print(f" Realisation {i+1}: t* = {tstar:.3f}")

```

```
print("\nBreakout times t* for log-normal model:")
for i, tstar in enumerate(normal_breakout_times):
    print(f" Realisation {i+1}: t* = {tstar:.3f}")
```



Breakout times t^* for log-uniform model:

```
Realisation 1: t* = 8.500
Realisation 2: t* = 11.420
Realisation 3: t* = 1.990
Realisation 4: t* = 3.510
Realisation 5: t* = 12.780
```

Breakout times t^* for log-normal model:

```
Realisation 1: t* = 17.640
Realisation 2: t* = 12.660
Realisation 3: t* = 0.860
Realisation 4: t* = 3.240
Realisation 5: t* = 2.160
```

Exercise 3.5.

In this exercise, we will use the log-uniform model. We will draw N independent samples of a and hence x . For $x(t)$ at $t = \frac{1}{5}$ and t_* , we must estimate the means and standard deviations and produce histograms for them. We use the

`log_uniform` and `build_field` methods to generate 2000 realisations and build an interpolated $\kappa(x)$ field.

```
In [127... log_uniform_samples = np.array([log_uniform(grid, modes) for _ in range(2000)])
kappa_fields = [build_field(grid, sample) for sample in log_uniform_samples]

# call simulate function
trajectories, breakout_times = simulate(kappa_fields)
# extract x values at x_T -> t = 0.2 and breakout times
x_T = np.array([np.interp(0.2, t, x) for (t, x) in trajectories])
```

```

breakout_times = np.array(breakout_times)

# generate summary statistics
mean_x_T = np.mean(x_T)
stdev_x_T = np.std(x_T)
mean_t_star = np.mean(breakout_times)
stdev_t_star = np.std(breakout_times)

print(f"Mean of  $x(1/5)$ : {mean_x_T:.4f}, Standard Deviation of  $x(1/5)$ : {stdev_x_T:.4f}")
print(f"Mean of  $t_*$ : {mean_t_star:.4f}, Standard Deviation of  $t_*$ : {stdev_t_star:.4f}")

# generate histograms
plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.hist(x_T, bins=30, edgecolor='black')
plt.xlabel(r"Particle position  $x(t)$  at  $t=\frac{1}{5}$ ")
plt.ylabel("Frequency")
plt.title(r"2007942 - Distribution of  $x(\frac{1}{5})$  under the log-uniform model")
plt.grid(True)

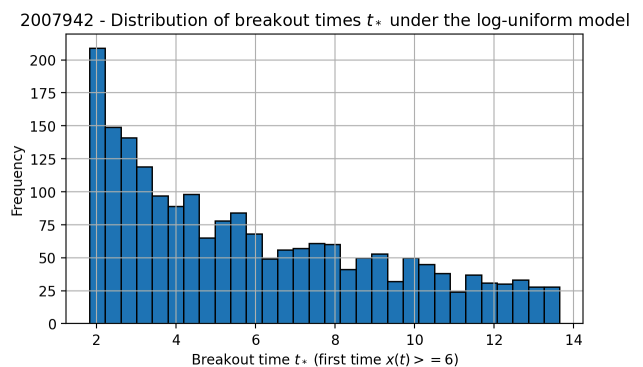
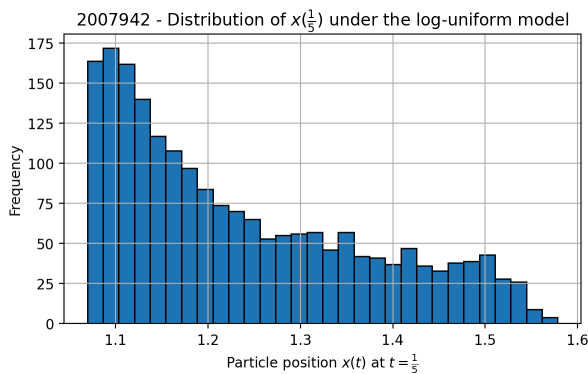
plt.subplot(1,2,2)
plt.hist(breakout_times, bins=30, edgecolor='black')
plt.xlabel(r"Breakout time  $t_*$  (first time  $x(t) \geq 6$ )")
plt.ylabel("Frequency")
plt.title(r"2007942 - Distribution of breakout times  $t_*$  under the log-uniform model")
plt.grid(True)

plt.tight_layout()
plt.show()

```

Mean of $x(1/5)$: 1.2357, Standard Deviation of $x(1/5)$: 0.1341

Mean of t_* : 5.9083, Standard Deviation of t_* : 3.2941



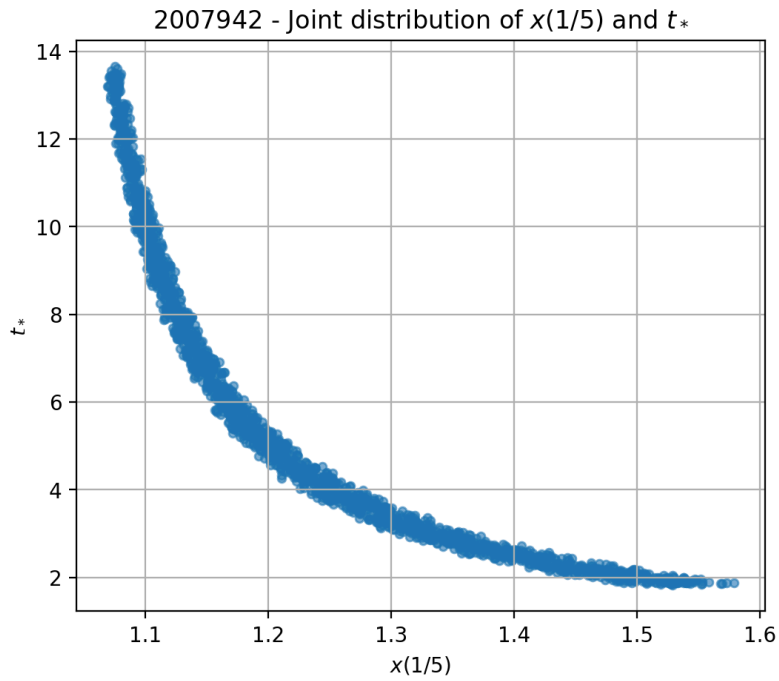
The distribution of $x(1/5)$ histogram is right-skewed for $t = \frac{1}{5}$, showing that particles are generally close to the starting point at $x \approx 1.1$. For the breakout time histogram, it is also right-skewed. This means the majority of particles reach the boundary condition $x = 6$ between $t = 2$ and $t = 4$ seconds.

In [128...]

```

plt.figure(figsize=(6,5))
plt.scatter(x_T, breakout_times, s=15, alpha=0.6)
plt.xlabel(r" $x(1/5)$ ")
plt.ylabel(r" $t_*$ ")
plt.title(r"2007942 - Joint distribution of  $x(1/5)$  and  $t_*$ ")
plt.grid(True)
plt.show()

```



The joint distribution of $x(1/5)$ and t_* demonstrates a decreasing non-linear relationship. This translates to particles that move faster earlier on reach the target sooner. The joint mean lies outside the support of the joint distribution. For the simulation, the sample size $N = 2000$ was used.

A sample size of $N = 2000$ was used as a compromise of variance and computational resource allocation. Most modern laptops can handle $N = 2000$ in under a minute of simulation. In Monte Carlo simulations, we can quantify the standard error of the sample mean by the formula

$$SE(X) \approx \frac{\sigma}{\sqrt{N}}$$

Given that $\text{Std}[x(1/5)] \approx 0.134$ and $\text{Std}[t_*] \approx 3.29$, applying the formula gives

$$SE_{x(1/5)} = \frac{0.134}{\sqrt{2000}} \approx 0.003.$$

$$SE_{t_*} = \frac{3.29}{\sqrt{2000}} \approx 0.073.$$

With the standard error being so small, it further shows that $N = 2000$ was an appropriate sample size.